RELATÓRIO DE AULA - TVI, BOLZANO E MÉTODO DA BISSEÇÃO

Disciplina: *Cálculo Numérico*
Aluna: *Victoria Antonia Brito Barbosa*
Turma: *NB*
Curso: *Bacharelado em Ciência e Tecnologia - UNIFESP*
Ambiente: *Google Colab, Python e GeoGebra*

1. Introdução

Nesta aula, estudei o Teorema do Valor Intermediário (TVI), o Teorema de Bolzano e o método da bisseção, compreendendo como a continuidade de uma função permite garantir a existência de raízes e como podemos aproximá-las numericamente.

Utilizei o GeoGebra para visualizar as funções e acompanhar as iterações, além do Python para automatizar os cálculos e analisar a precisão dos resultados.

2. Objetivos

- Compreender os teoremas do Valor Intermediário e de Bolzano.
- Identificar as condições de existência de raízes.
- Entender e aplicar o método da bisseção.
- Utilizar GeoGebra e Python para realizar aproximações numéricas.
- Analisar o controle do erro e a influência da tolerância.

3. Teorema do Valor Intermediário e Teorema de Bolzano

O Teorema do Valor Intermediário estabelece que uma função contínua em [a, b] assume todos os valores entre f(a) e f(b).

O Teorema de Bolzano é um caso particular, utilizado para identificar raízes. Se uma função é contínua no intervalo e apresenta sinais opostos nas extremidades, então existe pelo menos uma raiz em seu interior.

A condição é:

$f(a) \cdot f(b) < 0$

É importante destacar que a mudança de sinal garante a existência de uma raiz quando há continuidade, mas não garante sua unicidade.

Como exemplo, considerei:

$f(x) = x^3 - x - 2$

No intervalo [1, 2]:

- f(1) = -2
- f(2) = 4

Como existe mudança de sinal e a função é polinomial, portanto contínua, existe pelo menos uma raiz nesse intervalo.

4. Método da bisseção

O método da bisseção permite aproximar raízes dividindo sucessivamente um intervalo ao meio.

Em cada iteração, calculamos:

$m = (a + b) / 2$

Depois, verificamos em qual metade permanece a mudança de sinal e mantemos somente essa parte do intervalo.

A cada iteração, a largura do intervalo é reduzida pela metade. Assim, a aproximação da raiz se torna progressivamente mais precisa.

5. Atividades práticas no GeoGebra

5.1. Atividade 4 - Teorema do Valor Intermediário

Utilizei a função g(x) = x² + 1 no intervalo [-1, 2], verificando os valores das extremidades e construindo a reta horizontal y = 3.

A interseção ocorre em c = √2 ≈ 1,41 dentro do intervalo, demonstrando graficamente o Teorema do Valor Intermediário.

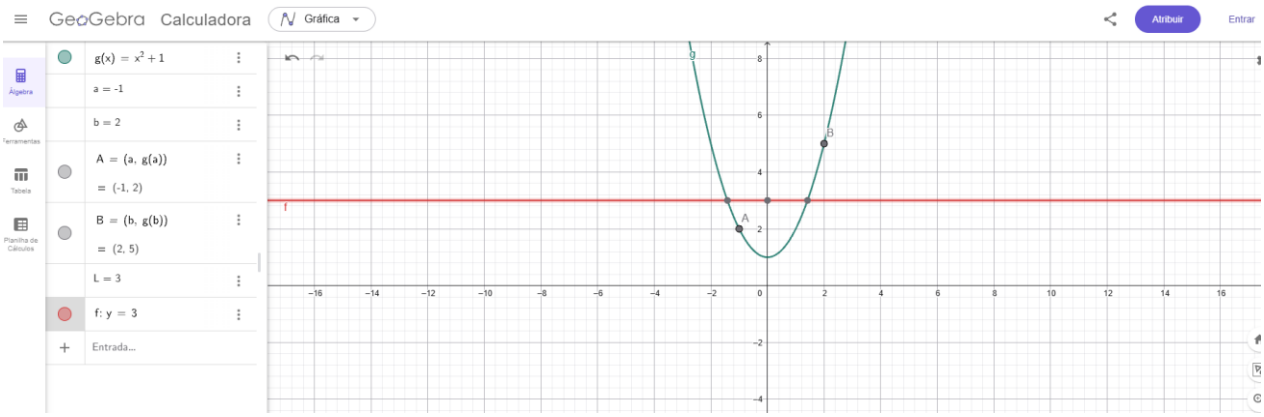


Figura 1 — Visualização do Teorema do Valor Intermediário no GeoGebra.

5.2. Atividade 5 — Bolzano e bisseção

Utilizei a função f(x) = x³ - x - 2 no intervalo [1, 2]. Verifiquei a mudança de sinal e realizei três iterações do método da bisseção:

Iteração	Ponto médio	Intervalo seguinte
1	1,5	[1,5; 2]
2	1,75	[1,5; 1,75]
3	1,625	[1,5; 1,625]

A atividade permitiu acompanhar a redução do intervalo e a aproximação da raiz.

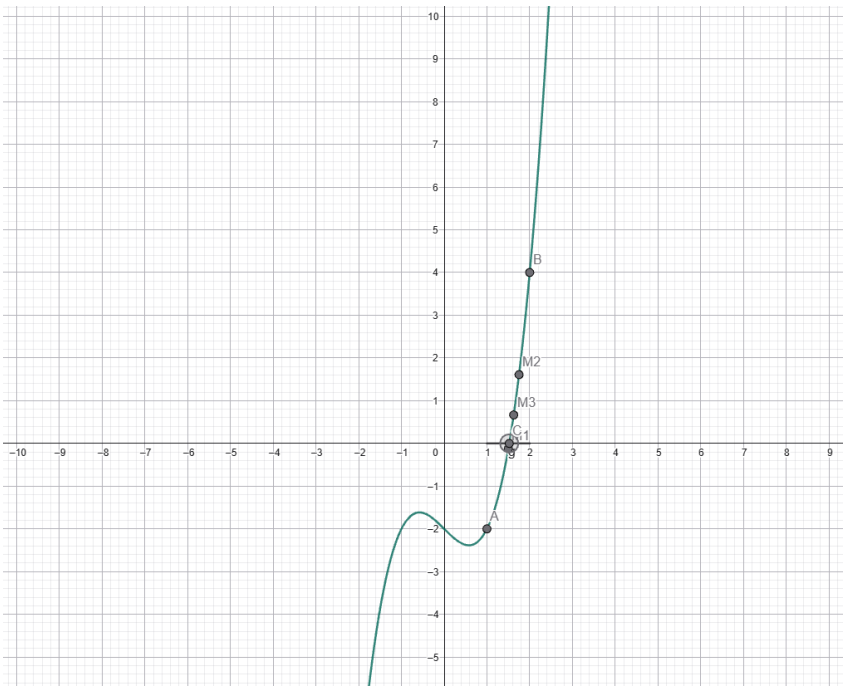


Figura 2 — Aplicação do método da bisseção no GeoGebra.

5.3. Atividade 9 - Aplicação à função cosseno

Utilizei a função h(x) = cos(x) - x no intervalo [0, 1].

Verifiquei que h(0) = 1 e h(1) ≈ -0,4597. Como existe mudança de sinal e a função é contínua, Bolzano garante pelo menos uma raiz no intervalo.

Realizei quatro iterações manuais:

Iteração	Ponto médio	Intervalo seguinte
1	0,5	[0,5; 1]
2	0,75	[0,5; 0,75]
3	0,625	[0,625; 0,75]
4	0,6875	[0,6875; 0,75]

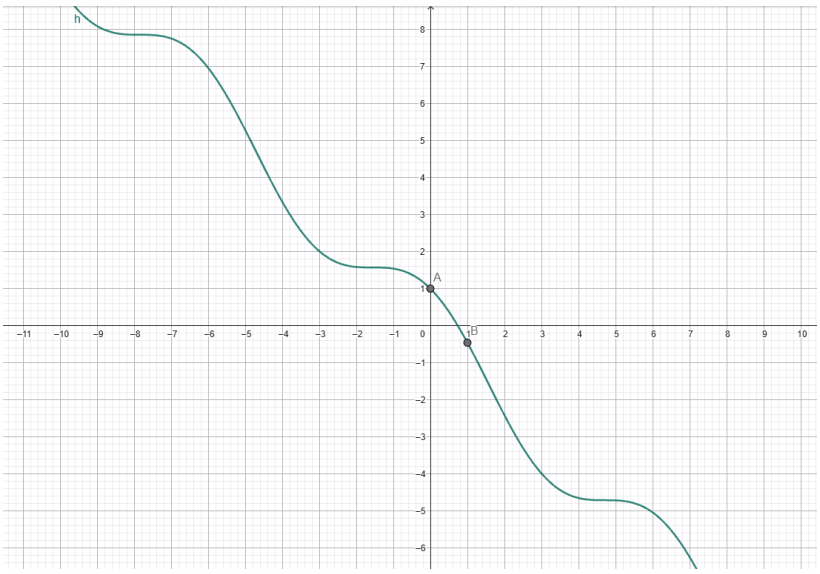


Figura 3 — Aplicação de Bolzano e primeiras iterações da bisseção no GeoGebra.

6. Implementação do método em Python

Após compreender o procedimento graficamente, implementei o método da bisseção em Python.

O algoritmo verifica as condições iniciais, calcula os pontos médios, seleciona os novos intervalos e registra o histórico das iterações. Também utiliza critérios de parada para controlar a precisão e evitar cálculos desnecessários.

Essa implementação permitiu automatizar o processo e comparar os resultados com as aproximações realizadas manualmente no GeoGebra.

7. Número de iterações e controle do erro

A cada iteração, a largura do intervalo é reduzida pela metade, conforme:

$$L_n = L_0 / 2^n$$

Quanto menor a tolerância escolhida, maior será a quantidade de iterações necessária para atingir a precisão desejada.

Também compreendi que a largura do intervalo, sua metade e o valor absoluto da função no ponto médio são critérios diferentes de controle do erro.

8. Testes das hipóteses de Bolzano

Analisei situações que demonstram a importância das condições do teorema:

- **Raiz sem mudança de sinal:** a função $p(x) = (x - 1)^2$ possui uma raiz, mesmo sem apresentar sinais opostos nas extremidades.
- **Mudança de sinal sem continuidade:** a função $q(x) = 1/(x - 1)$ apresenta uma descontinuidade em $x = 1$, portanto não permite aplicar Bolzano no intervalo [0, 2].
- **Produto nulo:** quando uma extremidade já apresenta valor zero, ela própria é uma raiz.

Esses exemplos demonstraram que a continuidade e a mudança estrita de sinal precisam ser verificadas corretamente.

9. Atividade com Python - h(x) = cos(x) - x

Na segunda parte da atividade 9, implementei a verificação de Bolzano e apliquei o método da bisseção com as tolerâncias 10^{-2}, 10^{-4} e 10^{-8}.

Os resultados foram organizados em uma tabela com a raiz aproximada, o valor absoluto da função e o número de iterações.

A implementação foi desenvolvida à minha maneira, aproveitando os códigos construídos anteriormente no notebook, e apresentou resultados coerentes com o esperado.

Foi possível observar que tolerâncias menores proporcionam maior precisão, mas exigem mais iterações e, consequentemente, maior custo computacional.

10. Conclusão

Nesta aula, compreendi a relação entre continuidade, existência de raízes e métodos numéricos de aproximação. As atividades no GeoGebra permitiram

visualizar os conceitos e acompanhar as iterações da bisseção, enquanto o Python possibilitou automatizar os cálculos e comparar diferentes níveis de precisão.

Concluo que o método da bisseção é uma técnica simples e confiável para aproximar raízes, desde que as condições do Teorema de Bolzano sejam respeitadas. Também compreendi a importância da escolha da tolerância no equilíbrio entre precisão e custo computacional.